Project report on

AUDIO PLAYER USING ESP32

Submitted by:

1. Sai Srujitha BT23ECI042

A report submitted for the partial fulfilment of the requirements of the course
ECE-207 Workshop - II

Submission Date: 26/09/2024

Task Number # 2

Under the guidance of:
Dr. Mayank Thacker
Department of Electronics and Communication Engineering

भारतीय सूचना प्रौद्योगिकी संस्थान, नागपुर
Indian Institute of Information Technology, Nagpur

Table of content

Chapter 1: Introduction

This project involves creating a simple **audio player** using the **ESP32**, a powerful microcontroller with built-in Wi-Fi and Bluetooth capabilities. The ESP32 is programmed to serve a web-based interface, allowing users to control the playback of an audio file, adjust the volume, and modify bass settings remotely through a web browser.

The primary goal of this project is to leverage the **ESP32's Wi-Fi functionality** to create an **IoT-enabled audio player**, where users can play, pause, and control the audio output from any device connected to the same network. The ESP32's **Digital-to-Analog Converter (DAC)** is used to output the audio signal to a connected speaker.

This project offers an innovative way to create a lightweight, versatile audio player using minimal external components. The removal of an SD card simplifies the design, making it more compact and efficient. Instead of relying on local storage, the ESP32 can stream audio from the web, or use its internal flash memory to store small audio files. Additionally, the user can control the system in real-time from a web interface, providing flexibility in adjusting playback settings such as volume and bass.

The integration of volume and bass control through the web interface demonstrates how even complex audio functionalities can be seamlessly managed through software, without adding extra hardware components. Additionally, the project can serve as a foundational platform for more advanced audio systems, offering potential future enhancements like real-time streaming from online sources, wireless audio synchronization, or integration with voice assistants.

Chapter 2: Circuit Design and Schematic

Components Needed:

1. **ESP32 Module**: The core of the project, responsible for web hosting, audio processing, and playback.
2. **Speaker**: For audio output, connected to the DAC pin of the ESP32.
3. **Amplifier (optional)**: To amplify the signal from the ESP32 DAC pin to drive larger speakers.
4. **Resistors (optional)**: For audio signal conditioning (depending on speaker requirements).
5. **Capacitors (optional)**: For filtering the audio signal (low-pass filtering for improved audio quality).

Pin Connections::

- **DAC Pin (GPIO 25)**: This pin outputs the audio signal. It will be connected to the amplifier or directly to a speaker.
- **GND (Ground)**: Connect to the ground of the speaker and power supply.
- **VCC (3.3V or 5V)**: Power the ESP32 using a 3.3V or 5V power supply, depending on the module you are using.

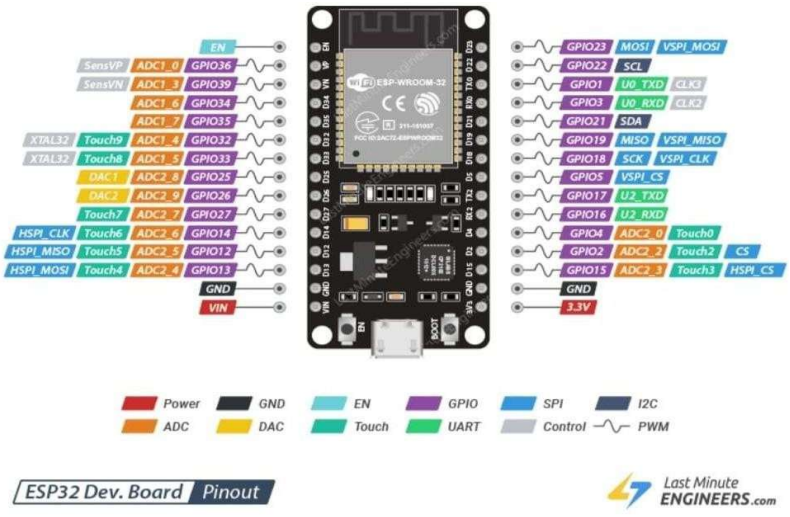


Fig.1: ESP32

Schematic Diagram:

1. **ESP32 to Speaker (Simple Setup)**:
   - Connect **GPIO 25** of the ESP32 directly to the positive terminal of the speaker.
   - Connect the negative terminal of the speaker to **GND**.
2. **ESP32 to Speaker with Amplifier (Optional)**:
   - Connect **GPIO 25** to the input pin of an audio amplifier module.
   - Connect the output pins of the amplifier to the speaker.
   - Power the amplifier with an external 5V source (depending on the module).

   This schematic provides the basic foundation for turning the ESP32 into an audio player, where the DAC pin generates the analog audio output signal based on the digital input file, ready to be amplified and played through a speaker.

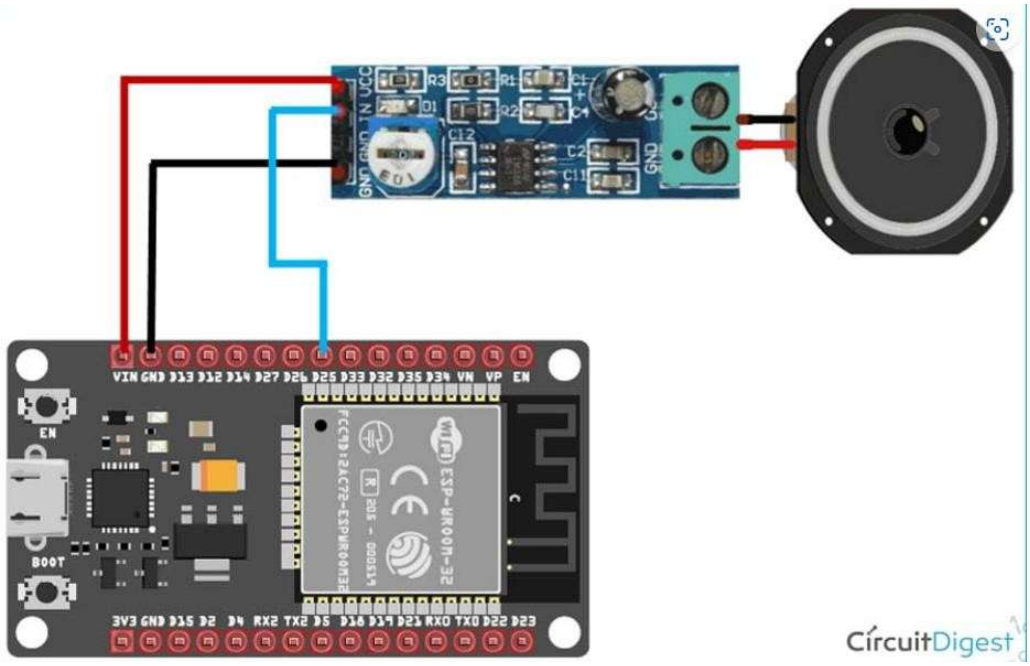


Fig 1: Connection diagram

Chapter 3: Code:

```
#include <WiFi.h>
#include <WebServer.h>
// WiFi credentials
const char* ssid = "LAPTOP-M67UPJHG 1811";
const char* password = "V5>92r57";

// Web server on port 80
WebServer server(80);

// Define audio data from the converted WAV file
const uint8_t audio_data[] = {
/* Insert your converted audio data array here */
};
const int audio_length = sizeof(audio_data) / sizeof(audio_data[0]);

// Variables for volume, bass control, and play/pause
int volume = 255;  // Max volume
int bass_level = 0;  // Bass level, default is 0 (no boost)
bool isPlaying = false;  // Playback state
// Previous sample for simple low-pass filter
float previous_sample = 0.0;

void handleRoot() {
// Send the web page with play/pause button, volume, and bass sliders
String html = "<html>\
<head>\
<title>ESP32 Audio Player</title>\
```

```
  <meta name='viewport' content='width=device-width, initial-scale=1'>\
<style>\
  body { \
  font-family: Arial, sans-serif;
  \ text-align: center; \
  padding: 20px; \
  background: url('https://your-image-url.com/background.jpg') no-repeat center center fixed;
  \ background-size: cover; \
  color: white; \
  }\
  h2 { \
  color: #ffcc00; \
  text-shadow: 2px 2px 8px #000000; \
  }\
  .slider { \
  width: 300px; \
  margin: 20px; \
  }\
  button { \
  padding: 15px 30px; \
  font-size: 18px; \
  margin: 20px; \
  border: none; \
  background: #ffcc00;
  \ color: #fff; \
  border-radius: 5px; \
  box-shadow: 2px 2px 10px rgba(0,0,0,0.3); \
  transition: background 0.3s ease, transform 0.3s ease; \
  cursor: pointer; \
  }\
```

```
button:hover { \
  background: #ff9900; \
  transform: scale(1.1); \
  }\
button:active { \
  transform: scale(0.95); \
  }\
  input[type='range'] { \
  -webkit-appearance: none; \
  appearance: none; \
  width: 100%; \
  height: 10px; \
  background: #333;
  \ border-radius:
  5px; \ outline:
  none; \ opacity:
  0.7; \
  transition: opacity 0.2s ease; \
  }\
  input[type='range']::-webkit-slider-thumb { \
  -webkit-appearance: none; \
  appearance: none; \
  width: 25px; \
  height: 25px; \
  background: #ffcc00;
  \ cursor: pointer; \
  border-radius: 50%; \
  }\
input[type='range']:hover { \
opacity: 1; \}\
```

```
.container { \ margin-

 top: 50px; \

 }\

 </style
 >\

 </head
       >\

 <body
      >\

 <h2>ESP32 Audio Player</h2>\

 <div class='container'>\

 <button onclick='playAudio()'>Play</button>\

 <button onclick='pauseAudio()'>Pause</button>\

 <br>\

 <label for='volume'>Volume:</label>\

     <input type='range' id='volume' name='volume' min='0' max='255' value='" + String(volume) + "'
class='slider' onchange='setVolume(this.value)'>\

 <p>Volume: <span id='vol_val'>" + String(volume) + "</span></p>\

 <label for='bass'>Bass:</label>\

     <input type='range' id='bass' name='bass' min='0' max='100' value='" + String(bass_level) + "' class='slider'
onchange='setBass(this.value)'>\

 <p>Bass Level: <span id='bass_val'>" + String(bass_level) + "</span></p>\

 </div>\

 <script>\

 function setVolume(vol) {\

 var xhr = new XMLHttpRequest();\ xhr.open('GET',

 '/setVolume?vol=' + vol, true);\ xhr.send();\

 document.getElementById('vol_val').innerText = vol;\

 }\

 function setBass(bass) {\

 var xhr = new XMLHttpRequest();\ xhr.open('GET',

 '/setBass?bass=' + bass, true);\ xhr.send();\
```

```
document.getElementById('bass_val').innerText = bass;\
}\
function playAudio() {\
var xhr = new XMLHttpRequest();\
xhr.open('GET', '/play', true);\
xhr.send();\
}\
function pauseAudio() {\
var xhr = new XMLHttpRequest();\
xhr.open('GET', '/pause', true);\
xhr.send();\
}\
</script>\
</body>\
</html>";
server.send(200, "text/html", html);
}
void handleSetVolume()
{ if
(server.hasArg("vol")) {
volume = server.arg("vol").toInt();
Serial.println("Volume set to: " + String(volume));
server.send(200, "text/plain", "OK");
} else {
server.send(400, "text/plain", "Bad Request");
}
}
void handleSetBass() { if (server.hasArg("bass")) {

if (server.hasArg("bass")) {
```

```
  bass_level = server.arg("bass").toInt();
  Serial.println("Bass level set to: " + String(bass_level));
  server.send(200, "text/plain", "OK");
  } else {
  server.send(400, "text/plain", "Bad Request");
  }
  }

void handlePlay() {
  isPlaying = true;
  Serial.println("Playback started");
  server.send(200, "text/plain", "Playing");
  }

void handlePause() {
  isPlaying = false;
  Serial.println("Playback paused");
  server.send(200, "text/plain", "Paused");
  }

  // Apply a low-pass filter to boost bass
  uint8_t applyBassBoost(uint8_t
  sample) {
  // Simple low-pass filter with a configurable "bass_level"
  float alpha = bass_level / 100.0;  // Normalize bass level
  to 0-1 float current_sample = (float)sample;
  float filtered_sample = alpha * current_sample + (1 - alpha) * previous_sample;
  previous_sample = filtered_sample;
  return constrain((int)filtered_sample, 0, 255);
  }
```

```
void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid,
  password);
while (WiFi.status() != WL_CONNECTED) {
  delay(1000);
  Serial.println("Connecting to WiFi...");
  }
  Serial.println("Connected to WiFi");
  Serial.println("IP Address: " + WiFi.localIP().toString());

  server.on("/", handleRoot);
  server.on("/setVolume",
  handleSetVolume); server.on("/setBass",
  handleSetBass); server.on("/play",
  handlePlay); server.on("/pause",
  handlePause); server.begin();
  }
void loop() {
  server.handleCli
  ent(); if
  (isPlaying) {
  for (int i = 0; i < audio_length && isPlaying; i++) {
  uint8_t adjusted_sample = (audio_data[i] * volume) / 255;
  uint8_t bass_boosted_sample = applyBassBoost(adjusted_sample);
  dacWrite(25, bass_boosted_sample);
  delayMicroseconds(36.5);
  }
  }
  }
```

Chapter 4: Implementation and results:

1. **Setting Up the ESP32**:

- Start by flashing the ESP32 with the provided code, which sets up a web server that allows users to control the audio player via a browser.
- **Wi-Fi Setup**: ESP32 connects to a specified Wi-Fi network. Once connected, the IP address is displayed on the Serial Monitor. You can access the web interface by navigating to this IP in a browser.

2. **Web Interface**:

- The **web page** allows users to control playback (Play/Pause), adjust volume, and modify bass levels.
- The sliders for volume and bass control send requests to the ESP32 to adjust the playback parameters in real-time.
- When the "Play" button is pressed, the ESP32 reads the audio data and outputs it through the DAC pin to a speaker. The "Pause" button stops playback.

3. **Audio Processing**:

- The audio data is stored in an array (audio_data[]) on the ESP32.
- Volume adjustments are made by scaling the audio samples, while a simple low-pass filter boosts the bass.
- The audio signal is output via the **DAC pin (GPIO 25)**, and the timing is controlled using delayMicroseconds to maintain the correct audio sampling rate.

4. **Hardware Setup**:

- The DAC pin is connected to a small speaker or an amplifier for better audio quality.
- The circuit was powered through USB for development, with the ESP32 connected to a Wi-Fi network for remote control.

Results:

1. **Successful Web-Based Control**:

- The **web interface** worked seamlessly, allowing control over playback and audio settings like volume and bass.
- The interface was user-friendly, and changes made through the sliders (e.g., adjusting the volume or bass) were reflected immediately in the audio output.

2. **Audio Playback**:

- The ESP32 successfully played the audio file through the speaker, with volume and bass adjustments affecting the audio quality as expected.
- Playback was smooth, and there were no noticeable lags or interruptions when controlling the player through the web interface.

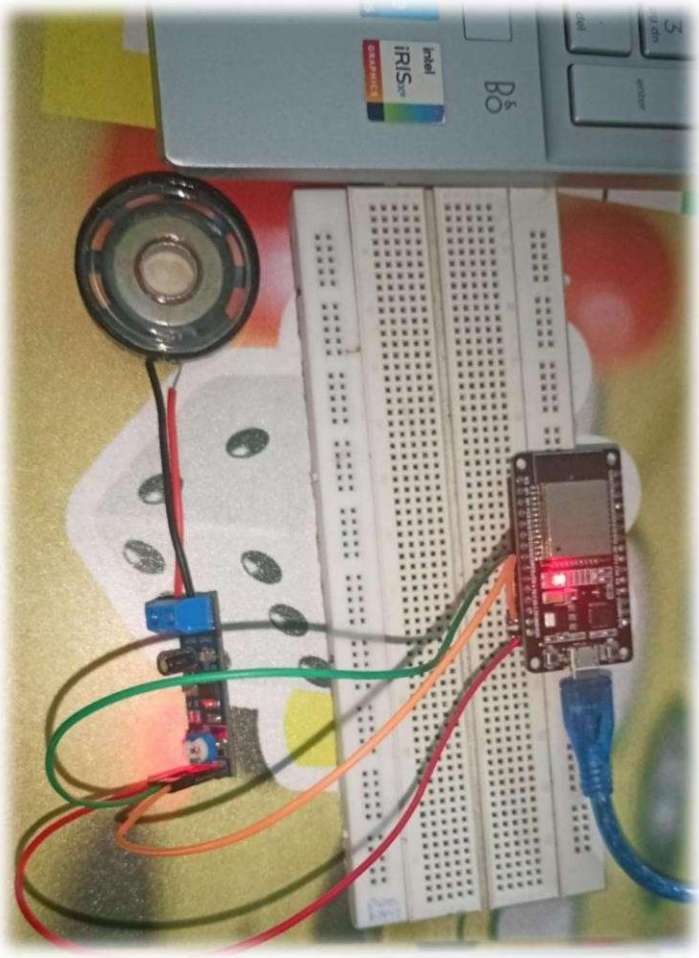
3. **Bass Boost and Volume Control**:

- The **bass boost** feature provided an improved low-frequency response, especially for small speakers. The low-pass filter allowed for some enhancement, making the audio fuller.
- Volume control worked effectively, scaling the output signal without introducing significant distortion.

4. **Wi-Fi and Network Response**:

- The **Wi-Fi connection** was stable, with minimal latency between sending commands from the web interface and receiving a response from the ESP32.
- The **response time** was quick, making the ESP32-based audio player feel responsive and user-friendly.


ESP32 Audio Player
Play
Pause
Volume:
Volume: 255
Bass:
Bass Level: 38


intel
iRIS

Chapter 5: Conclusion:

In this project, we successfully developed an ESP32-based audio player that can be controlled via a web interface. The implementation demonstrates the versatility of the ESP32, showcasing its ability to handle both **Wi-Fi communication** and **audio signal processing** using its built-in DAC.:

- **Web-Based Control**: The ESP32's integrated Wi-Fi module allows users to control the audio playback, volume, and bass levels wirelessly through an intuitive web interface. This highlights the ESP32's potential in IoT-based multimedia applications.
- **Audio Playback**: By leveraging the ESP32's DAC capabilities, we were able to convert digital audio data into analog signals, achieving decent sound output on a basic speaker system. The simple volume and bass control functions added a layer of user interactivity.
- **Real-Time Response**: The system's ability to respond quickly to commands sent from the web interface ensures a smooth and responsive user experience.

Overall, this project demonstrates the ESP32's potential for creating **low-cost, web-controlled multimedia devices**, combining networking, processing, and control in a single platform. Future improvements could involve better audio quality through external components, streaming capabilities, or integration with cloud- based services for dynamic audio content.

References:

1) https://circuitdigest.com/microcontroller-projects/building- diy-led-webserver-with-esp32

2) https://hackaday.io/project/187143-esp-32-based-audio-player